

Class 07: Machine Learning 1

Parisa Charoensook A17939591

Table of contents

Background	1
K - means clustering	1
Background	8
K - means clustering	8
Hierchial CLustering	16
Principal Component Analysis (PCA)	18
Analysis of UK food data	18
Data Import	18
Complete the following code to find out how many rows and columns are in x? . . .	19
Tidy Data	23
Exporatory Analysis	23
Digging Deeper (Variable Loadings)	28

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionallity reduction**.

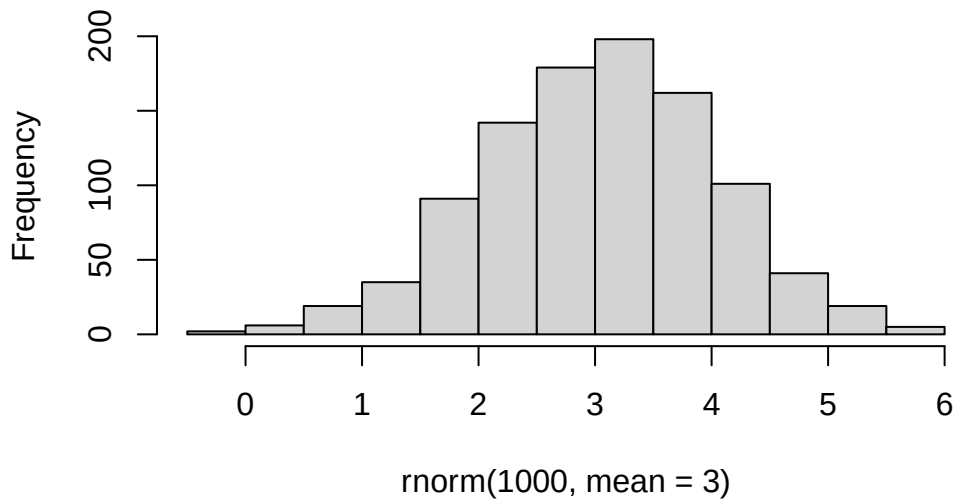
K - means clustering

The main function in “base” R for K- mmeans clustering is called `kmeans()`

Before we go too deep let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To do this we can use the `rnorm()` function:

```
hist(rnorm(1000, mean =3))
```

Histogram of rnorm(1000, mean = 3)



```
x <- c( rnorm(30, -3), rnorm(30, +3) )
x
```

```
[1] -2.5107592 -2.4692827 -3.8244174 -1.8758420 -3.9840539 -2.1591144
[7] -2.2026878 -2.9200720 -2.6777391 -2.4107758 -2.2539505 -2.8791159
[13] -4.1825400 -1.0817894 -2.9404422 -2.3442799 -1.6649317 -0.7908527
[19] -2.6353317 -4.2982260 -1.5717656 -4.1655004 -2.9814326 -2.0472704
[25] -4.0672861 -3.9282014 -3.4791268 -2.6976411 -3.1423684 -2.4959483
[31]  3.4373183  1.1844224  4.5830399  3.2469816  2.2409684  2.9078436
[37]  2.7524653  2.9364646  3.5822578  1.9994074  2.8229503  3.0196089
[43]  1.6745743  2.6546942  3.7985243  3.2441648  1.9824313  2.6618544
[49]  3.7364642  4.3247536  3.9752297  1.6713377  3.7555979  2.9355030
[55]  1.6845755  3.9080637  1.9807545  2.7369730  2.7750168  4.4595040
```

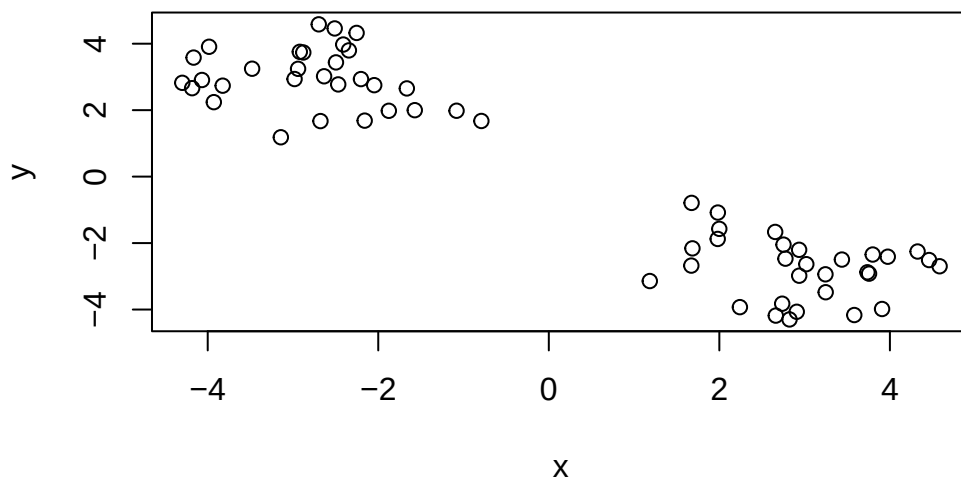
```
z <- cbind( x, y= rev(x))
z
```

	x	y
[1,]	-2.5107592	4.4595040
[2,]	-2.4692827	2.7750168
[3,]	-3.8244174	2.7369730
[4,]	-1.8758420	1.9807545

[5,]	-3.9840539	3.9080637
[6,]	-2.1591144	1.6845755
[7,]	-2.2026878	2.9355030
[8,]	-2.9200720	3.7555979
[9,]	-2.6777391	1.6713377
[10,]	-2.4107758	3.9752297
[11,]	-2.2539505	4.3247536
[12,]	-2.8791159	3.7364642
[13,]	-4.1825400	2.6618544
[14,]	-1.0817894	1.9824313
[15,]	-2.9404422	3.2441648
[16,]	-2.3442799	3.7985243
[17,]	-1.6649317	2.6546942
[18,]	-0.7908527	1.6745743
[19,]	-2.6353317	3.0196089
[20,]	-4.2982260	2.8229503
[21,]	-1.5717656	1.9994074
[22,]	-4.1655004	3.5822578
[23,]	-2.9814326	2.9364646
[24,]	-2.0472704	2.7524653
[25,]	-4.0672861	2.9078436
[26,]	-3.9282014	2.2409684
[27,]	-3.4791268	3.2469816
[28,]	-2.6976411	4.5830399
[29,]	-3.1423684	1.1844224
[30,]	-2.4959483	3.4373183
[31,]	3.4373183	-2.4959483
[32,]	1.1844224	-3.1423684
[33,]	4.5830399	-2.6976411
[34,]	3.2469816	-3.4791268
[35,]	2.2409684	-3.9282014
[36,]	2.9078436	-4.0672861
[37,]	2.7524653	-2.0472704
[38,]	2.9364646	-2.9814326
[39,]	3.5822578	-4.1655004
[40,]	1.9994074	-1.5717656
[41,]	2.8229503	-4.2982260
[42,]	3.0196089	-2.6353317
[43,]	1.6745743	-0.7908527
[44,]	2.6546942	-1.6649317
[45,]	3.7985243	-2.3442799
[46,]	3.2441648	-2.9404422
[47,]	1.9824313	-1.0817894

```
[48,] 2.6618544 -4.1825400
[49,] 3.7364642 -2.8791159
[50,] 4.3247536 -2.2539505
[51,] 3.9752297 -2.4107758
[52,] 1.6713377 -2.6777391
[53,] 3.7555979 -2.9200720
[54,] 2.9355030 -2.2026878
[55,] 1.6845755 -2.1591144
[56,] 3.9080637 -3.9840539
[57,] 1.9807545 -1.8758420
[58,] 2.7369730 -3.8244174
[59,] 2.7750168 -2.4692827
[60,] 4.4595040 -2.5107592
```

```
plot(z)
```



```
p <- 1:5
```

```
rev(p)
```

```
[1] 5 4 3 2 1
```

Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km <- kmeans(z , center = 2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

```
      x      y
1 -2.756092  2.955792
2  2.955792 -2.756092
```

Clustering vector:

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

Within cluster sum of squares by cluster:

```
[1] 48.08773 48.08773
(between_SS / total_SS =  91.1 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

```
attributes(km)
```

\$names

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

\$class

```
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What “component of your result object details cluster assignment / membership?

```
km$cluster
```

[illegible]

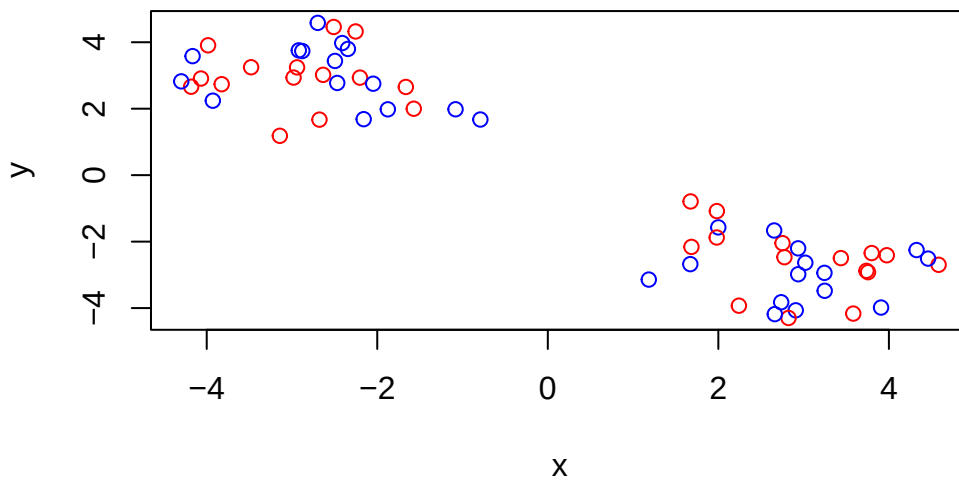
Q. Plot x colored by the kmeans cluster assignment and add cluster centers as blue points.

```
km$centers
```

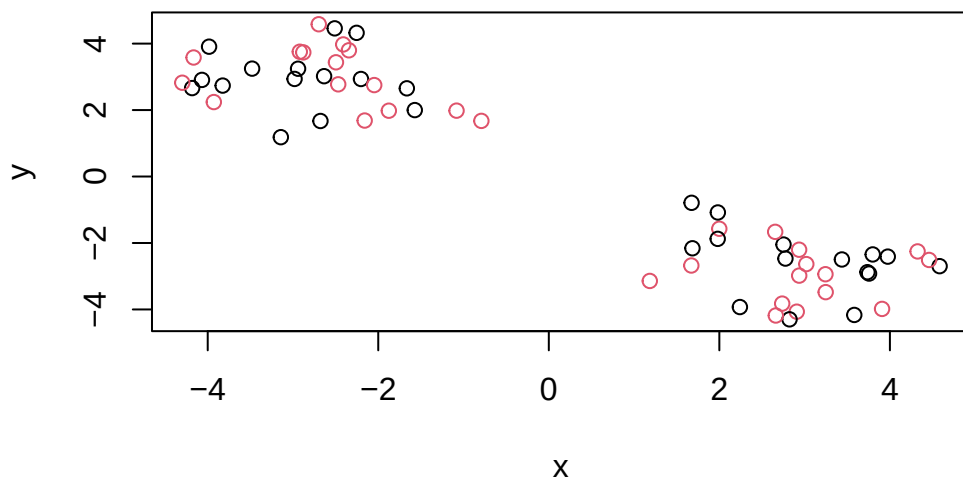
	x	y
1	-2.756092	2.955792
2	2.955792	-2.756092

Q. Plot $\mathbf{z}$ colored by the kmeans cluster assignment and add cluster centers as blue points.

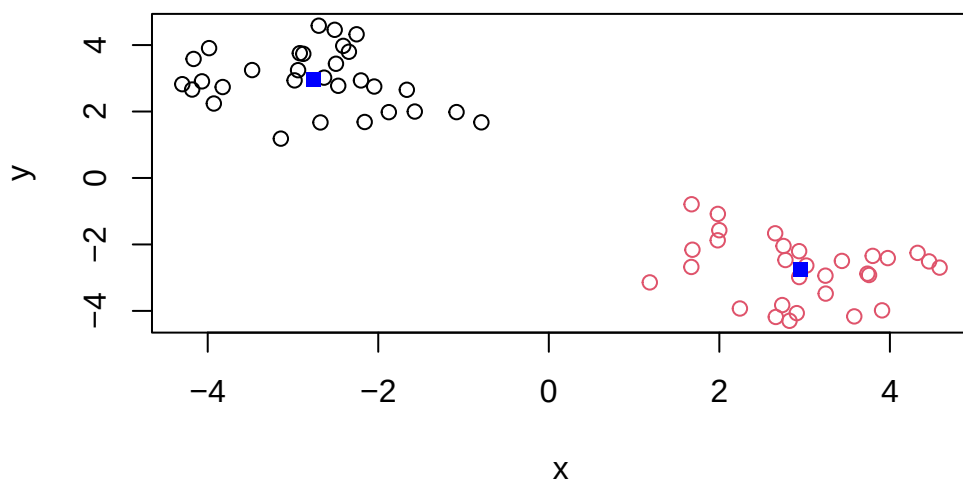
```
plot(z, col = c("red", "blue") )
```



```
plot (z, col=c(1,2))
```



```
plot(z, col = c(km$cluster))
points(km$centers, col = "blue", pch = 15 )
```



Background

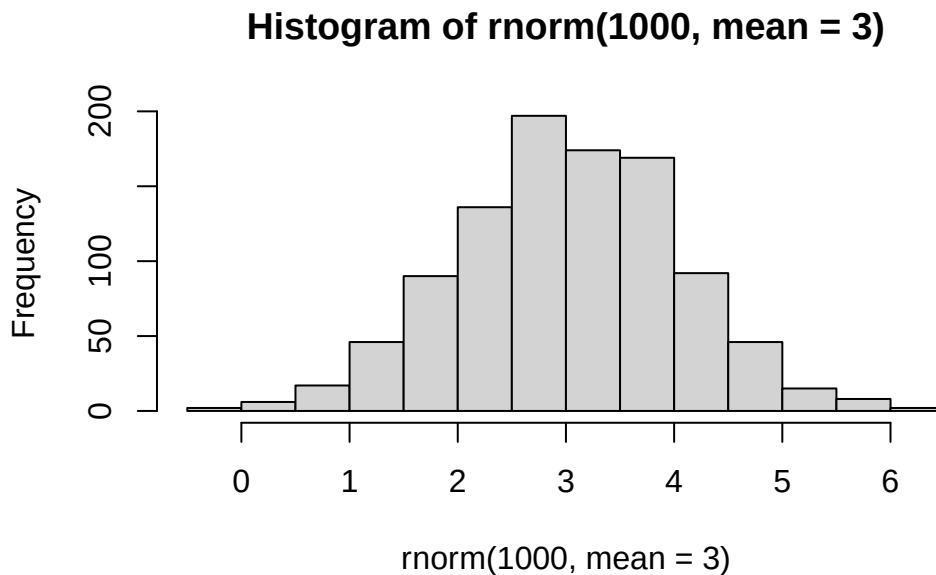
Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

K - means clustering

The main function in “base” R for K-means clustering is called `kmeans()`

Before we go too deep let’s make up some “simple” data that we can cluster and know if we are getting a good answer or not. To do this we can use the `rnorm()` function:

```
hist(rnorm(1000, mean = 3))
```



```
x <- c( rnorm(30, -3), rnorm(30, +3) )  
x
```

```
[1] -3.1192956 -2.7776336 -1.8640710 -2.1513342 -2.1954456 -2.2179715  
[7] -2.1025686 -1.4472443 -3.4109995 -3.0732131 -3.9418007 -2.6897429  
[13] -4.4586577 -3.0784933 -2.8047571 -3.7100474 -2.2688094 -1.1334799  
[19] -2.5202760 -3.1562655 -5.5396396 -2.5884165 -5.0915578 -1.8797636  
[25] -3.2331750 -3.1476097 -4.3870568 -2.7508915 -2.5422275 -3.0523456
```



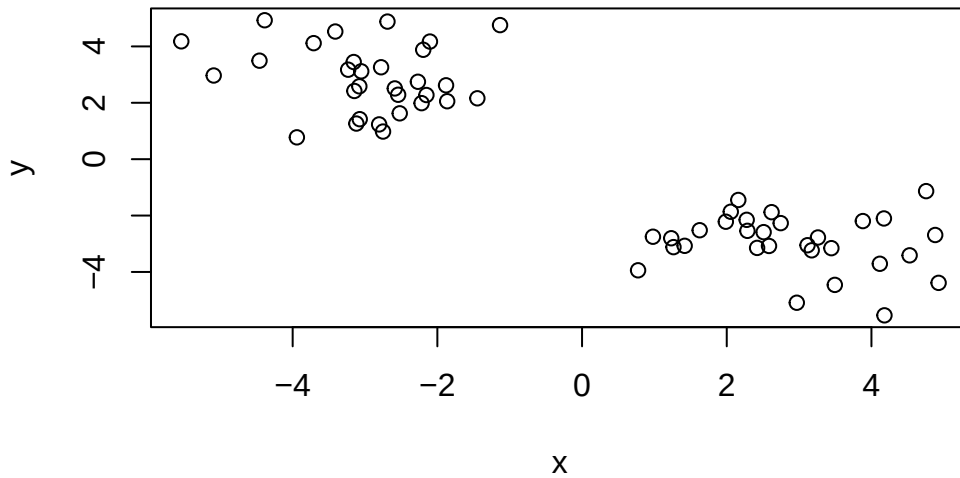
```
[31] 3.1163246 2.2855082 0.9796759 4.9283126 2.4196643 3.1741042
[37] 2.6198996 2.9680472 2.5096369 4.1796544 3.4461145 1.6259534
[43] 4.7556333 2.7459417 4.1157763 1.2330115 2.5839766 3.4937994
[49] 4.8811465 0.7742599 1.4183122 4.5276770 2.1596243 4.1723568
[55] 1.9871965 3.8811464 2.2756037 2.0557027 3.2605989 1.2636405
```

```
z <- cbind( x, y= rev(x))
z
```

```
      x      y
[1,] -3.1192956 1.2636405
[2,] -2.7776336 3.2605989
[3,] -1.8640710 2.0557027
[4,] -2.1513342 2.2756037
[5,] -2.1954456 3.8811464
[6,] -2.2179715 1.9871965
[7,] -2.1025686 4.1723568
[8,] -1.4472443 2.1596243
[9,] -3.4109995 4.5276770
[10,] -3.0732131 1.4183122
[11,] -3.9418007 0.7742599
[12,] -2.6897429 4.8811465
[13,] -4.4586577 3.4937994
[14,] -3.0784933 2.5839766
[15,] -2.8047571 1.2330115
[16,] -3.7100474 4.1157763
[17,] -2.2688094 2.7459417
[18,] -1.1334799 4.7556333
[19,] -2.5202760 1.6259534
[20,] -3.1562655 3.4461145
[21,] -5.5396396 4.1796544
[22,] -2.5884165 2.5096369
[23,] -5.0915578 2.9680472
[24,] -1.8797636 2.6198996
[25,] -3.2331750 3.1741042
[26,] -3.1476097 2.4196643
[27,] -4.3870568 4.9283126
[28,] -2.7508915 0.9796759
[29,] -2.5422275 2.2855082
[30,] -3.0523456 3.1163246
[31,] 3.1163246 -3.0523456
[32,] 2.2855082 -2.5422275
```

```
[33,] 0.9796759 -2.7508915
[34,] 4.9283126 -4.3870568
[35,] 2.4196643 -3.1476097
[36,] 3.1741042 -3.2331750
[37,] 2.6198996 -1.8797636
[38,] 2.9680472 -5.0915578
[39,] 2.5096369 -2.5884165
[40,] 4.1796544 -5.5396396
[41,] 3.4461145 -3.1562655
[42,] 1.6259534 -2.5202760
[43,] 4.7556333 -1.1334799
[44,] 2.7459417 -2.2688094
[45,] 4.1157763 -3.7100474
[46,] 1.2330115 -2.8047571
[47,] 2.5839766 -3.0784933
[48,] 3.4937994 -4.4586577
[49,] 4.8811465 -2.6897429
[50,] 0.7742599 -3.9418007
[51,] 1.4183122 -3.0732131
[52,] 4.5276770 -3.4109995
[53,] 2.1596243 -1.4472443
[54,] 4.1723568 -2.1025686
[55,] 1.9871965 -2.2179715
[56,] 3.8811464 -2.1954456
[57,] 2.2756037 -2.1513342
[58,] 2.0557027 -1.8640710
[59,] 3.2605989 -2.7776336
[60,] 1.2636405 -3.1192956
```

```
plot(z)
```



```
p <- 1:5
```

```
rev(p)
```

```
[1] 5 4 3 2 1
```

Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km <- kmeans(z , center = 2)
```

```
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	2.861277	-2.944493
2	-2.944493	2.861277

Clustering vector:

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Within cluster sum of squares by cluster:

```
[1] 70.70671 70.70671
(between_SS / total_SS = 87.7 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

```
attributes(km)
```

```
$names
```

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

```
$class
```

```
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What “component of your result object details cluster assignment / membership?

```
km$cluster
```

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

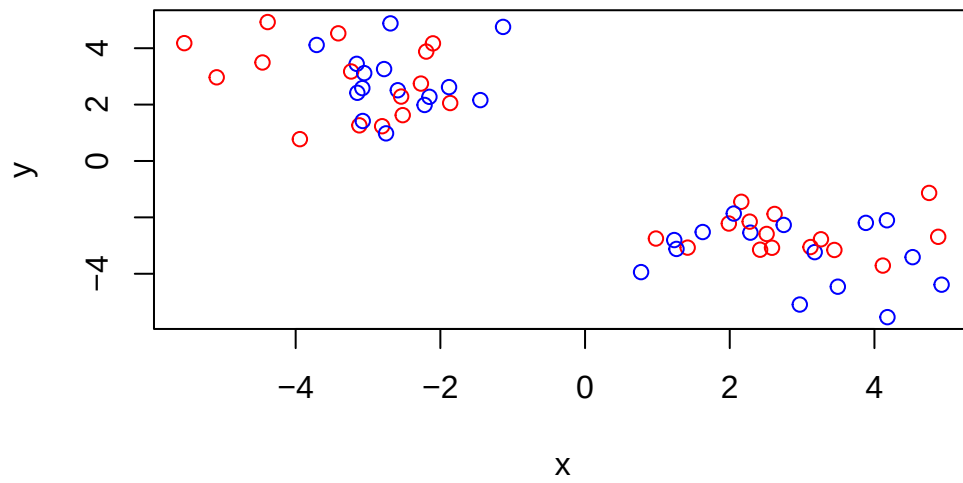
Q. Plot x colored by the kmeans cluster assignment and add cluster centers as blue points.

```
km$centers
```

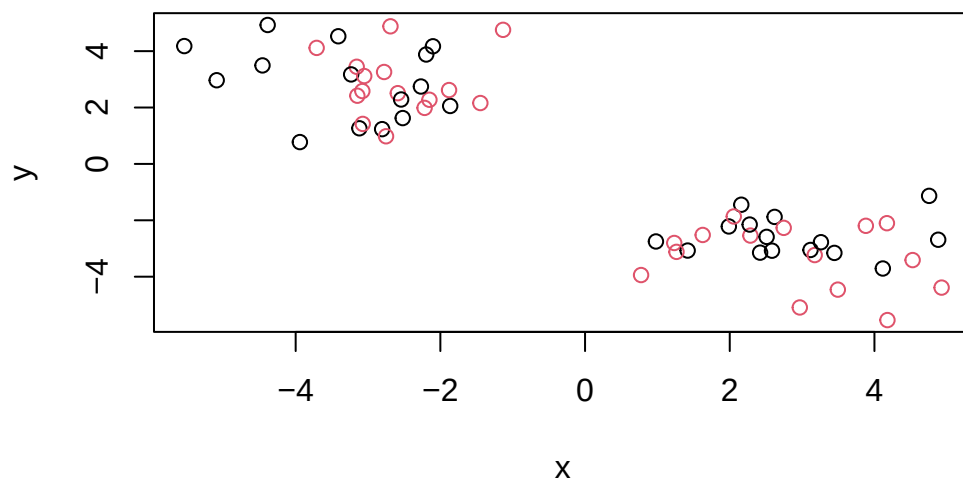
```
      x      y
1  2.861277 -2.944493
2 -2.944493  2.861277
```

Q. Plot z colored by the kmeans cluster assignment and add cluster centers as blue points.

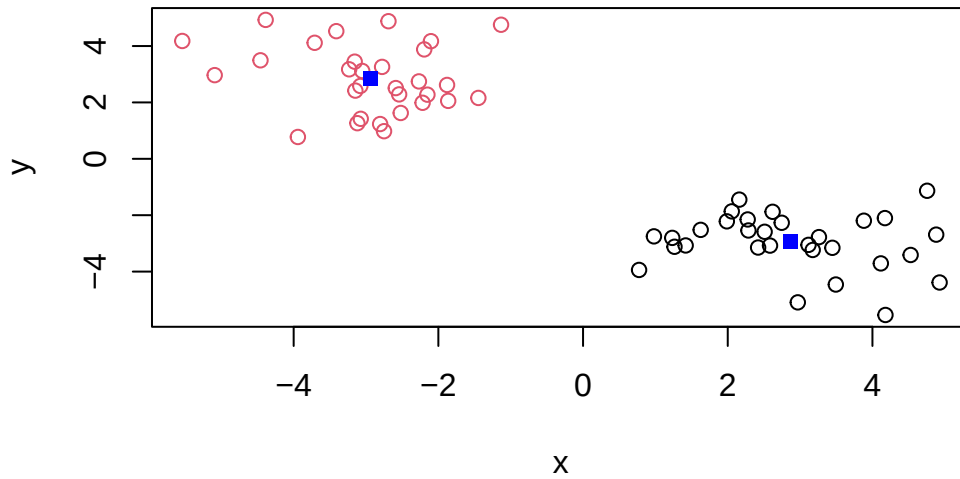
```
plot(z, col = c("red", "blue") )
```



```
plot (z, col=c(1,2))
```

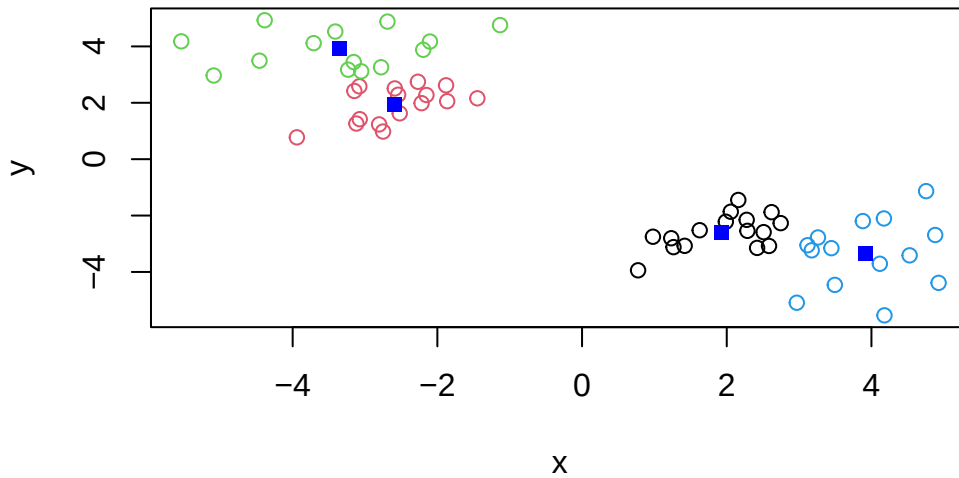


```
plot(z, col = c(km$cluster))
points(km$centers, col = "blue", pch = 15 )
```



Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4)?

```
km4 <- kmeans(z, centers = 4)
plot(z, col= km4$cluster)
points(km4$centers, col= "blue", pch=15)
```



**** N.B**** You need to tell K-means the number of clusters (i.e. set `centers = 2`)!!

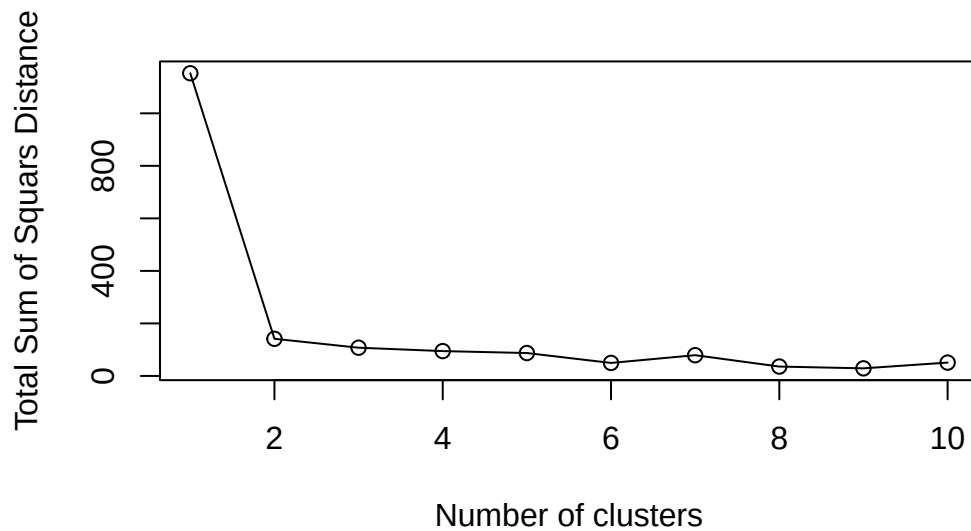
One approach is to try different values for `centers` and then pick the best...

```
km <- kmeans(z, centers = 5)
km$tot.withinss
```

```
[1] 66.78028
```

```
ans <- NULL
for (i in 1:10) {
  km <- kmeans(z, centers = i)
  ans <- c(ans, km$tot.withinss)
}

plot(ans, type = "o",
      xlab="Number of clusters", ylab= "Total Sum of Squares Distance")
```



Hierchial CLustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`

This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

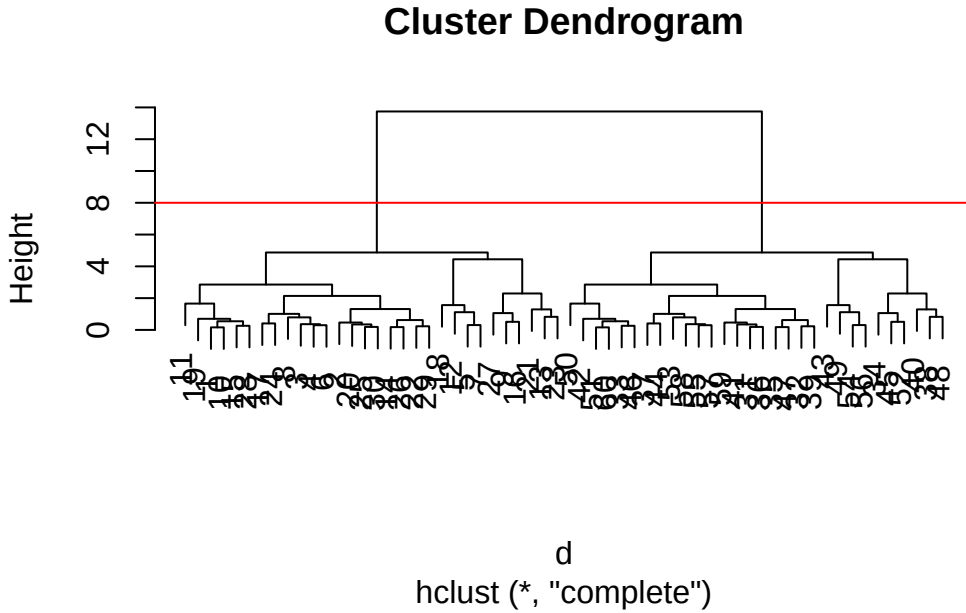
Call:

```
hclust(d = d)
```

```
Cluster method : complete
Distance       : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.


```
plot(hc)
abline(h=8, col = "red")
```



Once we have our `hclust` object (our “tree” of cluster dendrogram) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

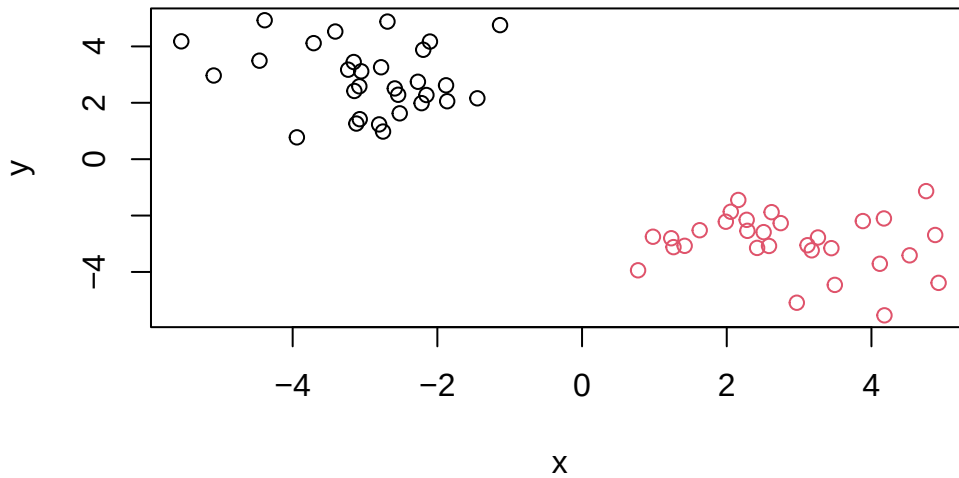
[illegible]

```
cutree(hc, k=4)
```

[1] 1 1 1 1 2 1 2 1 2 1 1 2 2 1 1 2 1 2 1 1 2 1 2 1 1 1 3 3 3 4 3 3 3 4
[39] 3 4 3 3 4 3 4 3 3 4 4 3 3 4 3 4 3 4 3 3 3 3

Q. Make a plot of $\mathbf{z}$ with your hclust results (i.e. colored by cluster membership)

```
grps <- cutree(hc, k=2)
plot(z, col=grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex dataset

Analysis of UK food data

Lets look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

The data is made avilable in CSV format, so we can see the (`read.csv`) function import to R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Q1. . How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

Complete the following code to find out how many rows and columns are in x?

```
____(x) ## [1] 17 5
```

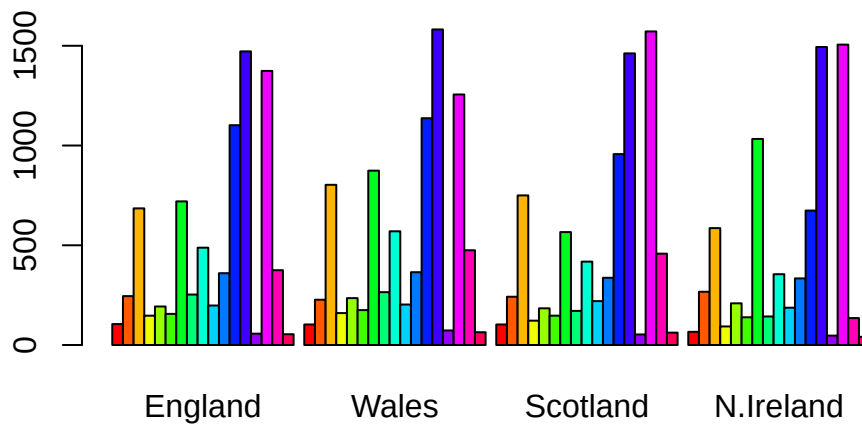
```
dim(x)
```

```
[1] 17  4
```

```
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



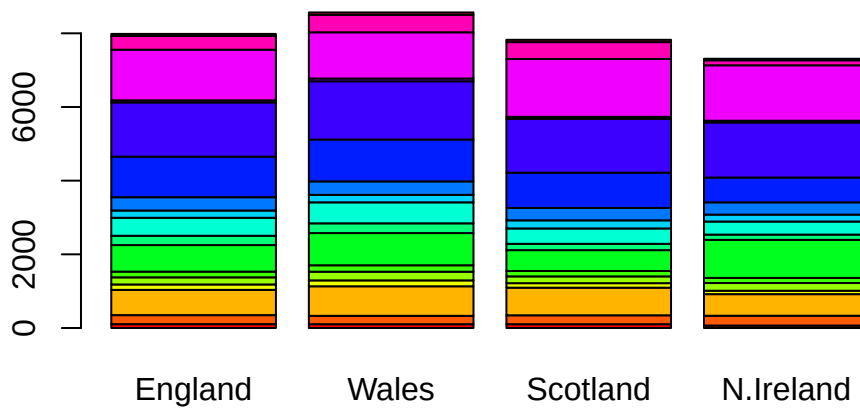
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

Using the second approach (`read.csv(..., rownames = 1)`) because it assigns the first column as row names during import and avoids extra steps for later. Instead of having to do multiple steps, which is faster, simpler and reduces the risks of mistakes.

Q3. Changing what optional argument in the above `barplot()` function results in the following plot?

This can be done by changing to `beside = FALSE`

```
barplot(as.matrix(x), beside=FALSE, col=rainbow(nrow(x)))
```



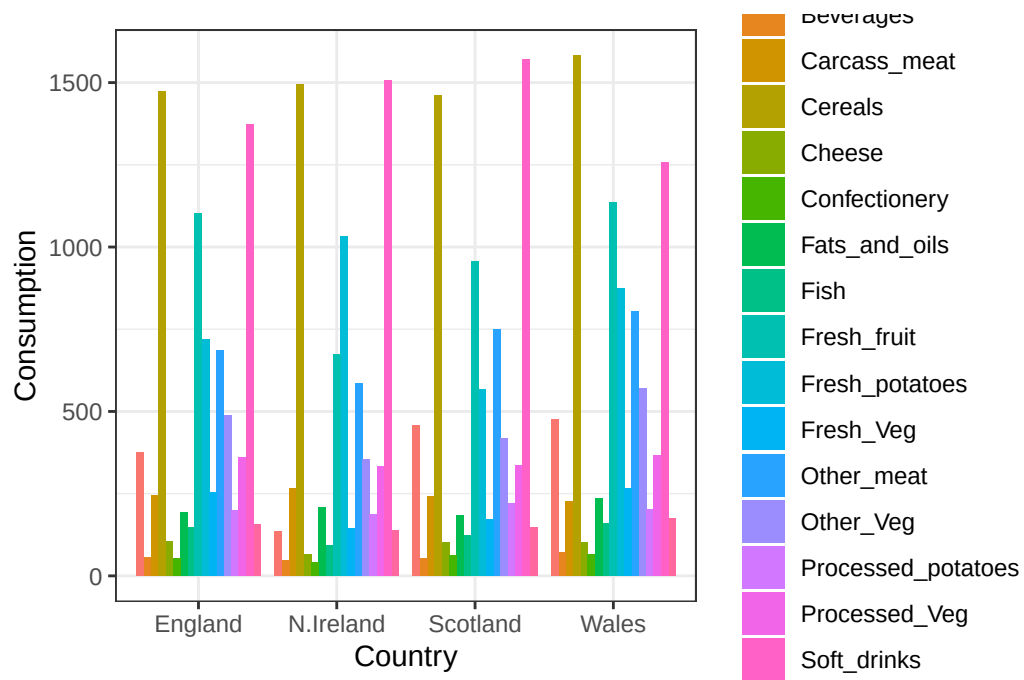
```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

```
[1] 68 3
```

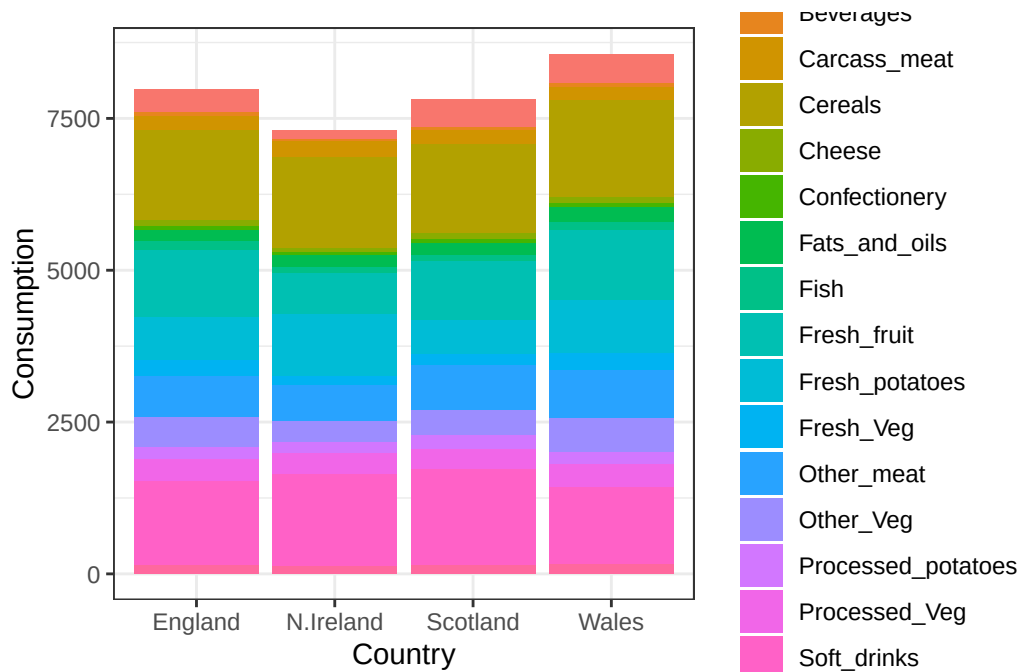
```
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4. Changing what optional argument in the above `ggplot()` code results in a stacked barplot figure?

By changing `dodge` to `stack` instead.

```
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



Tidy Data

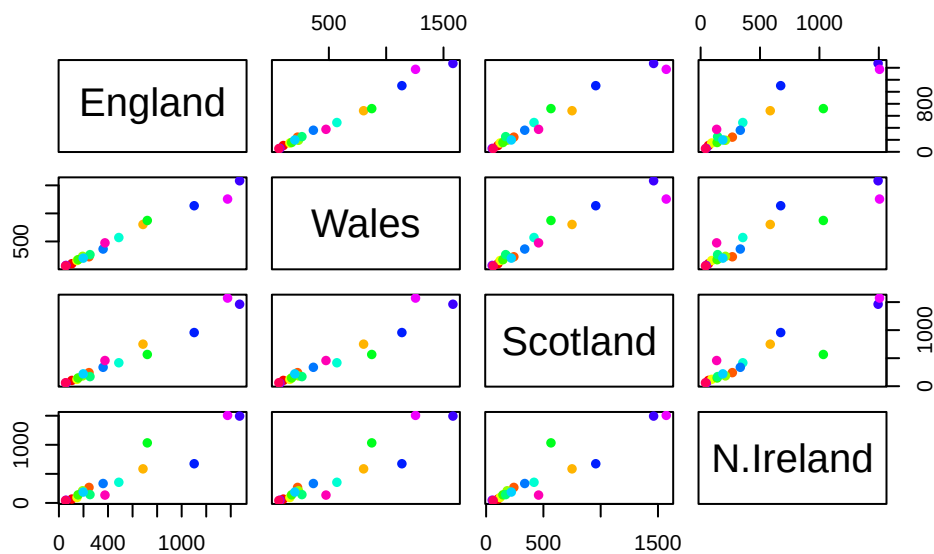
Exploratory Analysis

Make some plots to help make sense of obvious trends.

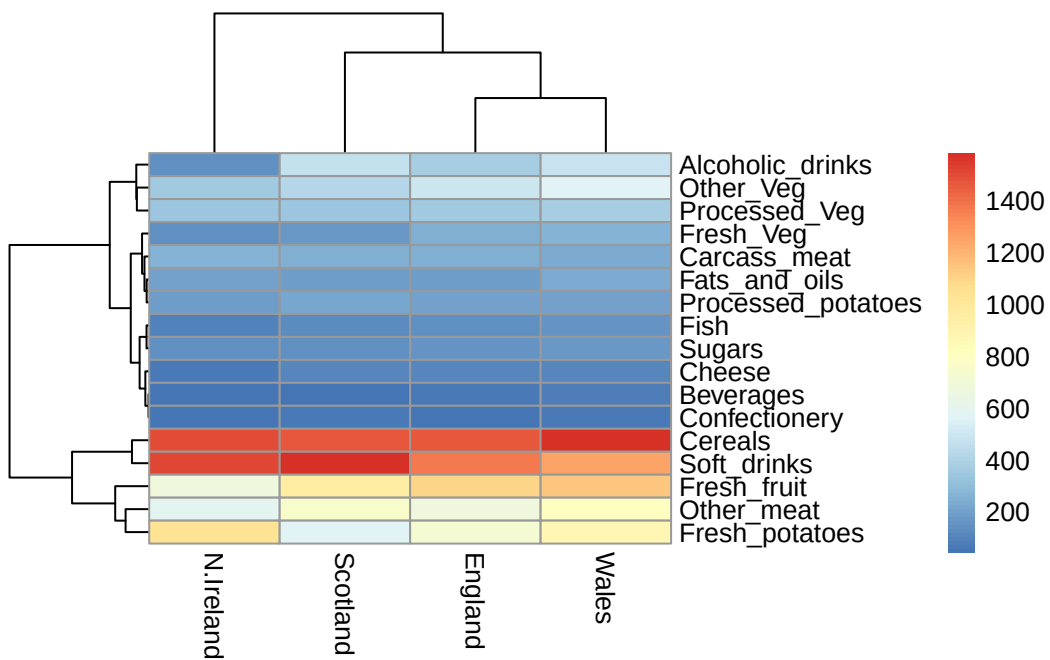
Q5. We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

By adding the `pairs()` function, it creates scatterplots comparing every pair of countries in the dataset. Each point represents one food category so the color helps distinguish different foods. This allows us to see food consumption patterns between countries. If a point lies on the diagonal for a given plot this means that the food category has the same consumption value in both countries being compared.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)
pheatmap( as.matrix(x) )
```



	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

England, Wales, and Scotland cluster more closely together so this tell us they have more similar food consumption patterns. N. Ireland appears more separated from the other countries, meaning its diet differs more from the rest of the UK in the dataset. The main differences for N. Ireland seem to be higher consumption of some foods such as carcass meat and lower consumption of others like fresh vegetables and fresh fruit.

Key-point: Even relatively small datasets can prove challenging to interpret.

The main function in “base” R for pca is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns. So here we need to take the transpose of our `x` input object.

```
pca <- prcomp( t(x) )
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

The returned `pca` object has component that we can use to make our main result figures:

```
attributes("pca")
```

NULL

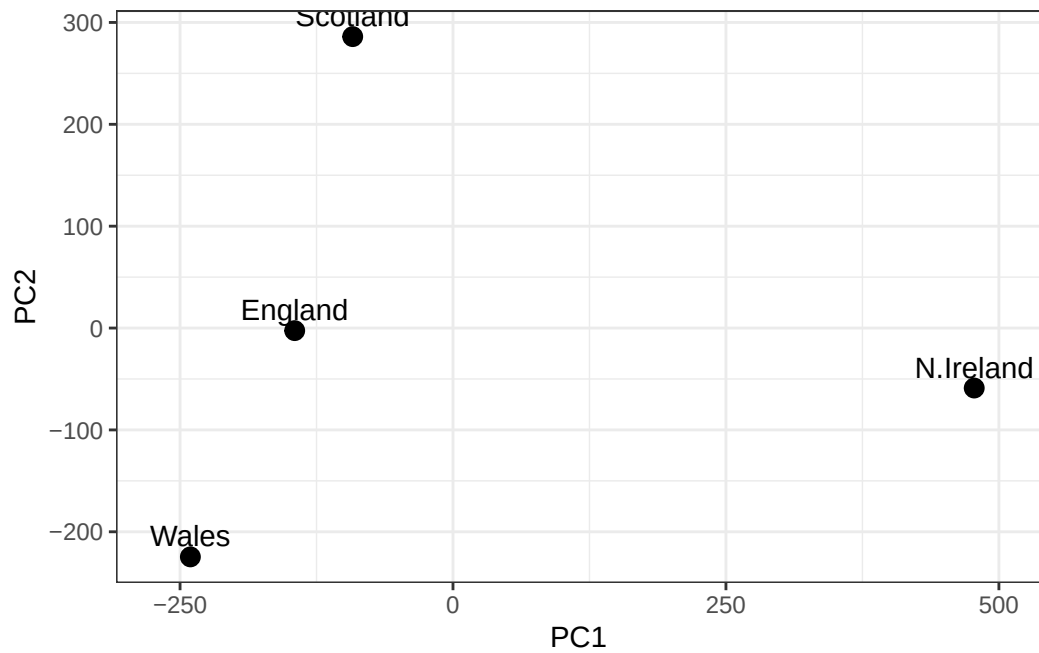
The main result figure from this analysis is called a “PC plot” or “ordination plot plot on pC1 vs PC2 plot”).

This plot shows how samples (in this case countries) relate to each other along our new PC axis.

This is our new “reduced-dimensional space”. In this case 2 dimensions, PC1 and PC2, that capture most of the variance in the original 17 dimensional data - set.

Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```

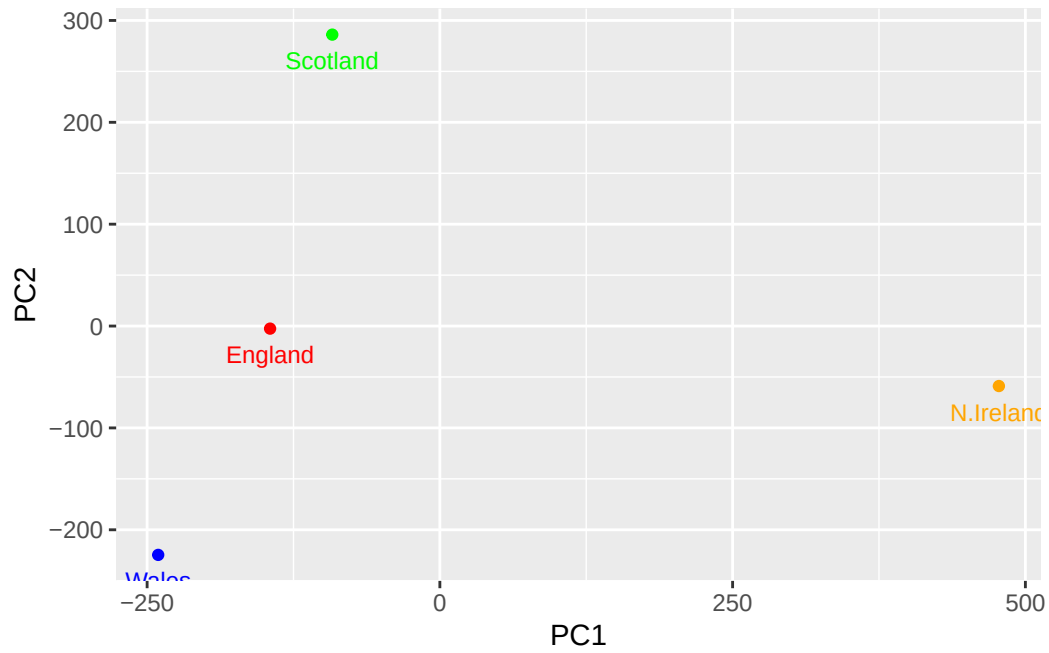


Q8. Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
library(ggplot2)

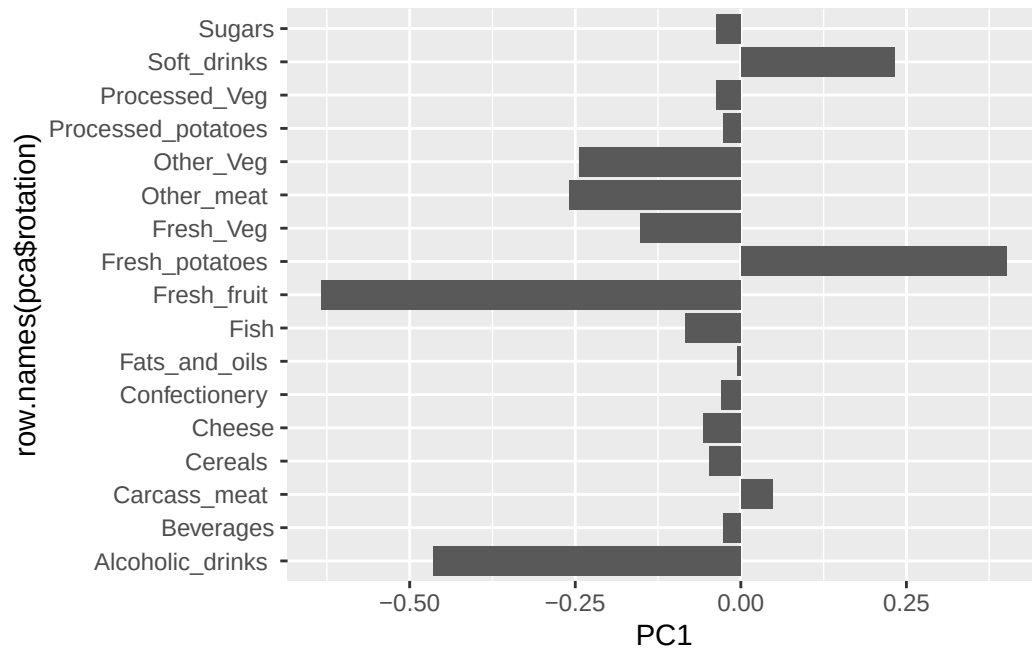
mycols <- c("red", "blue", "green", "orange")

ggplot(pca$x, aes(PC1, PC2, label = rownames(pca$x))) +
  geom_point(color = mycols) +
  geom_text(size = 3, vjust = 2, color = mycols)
```

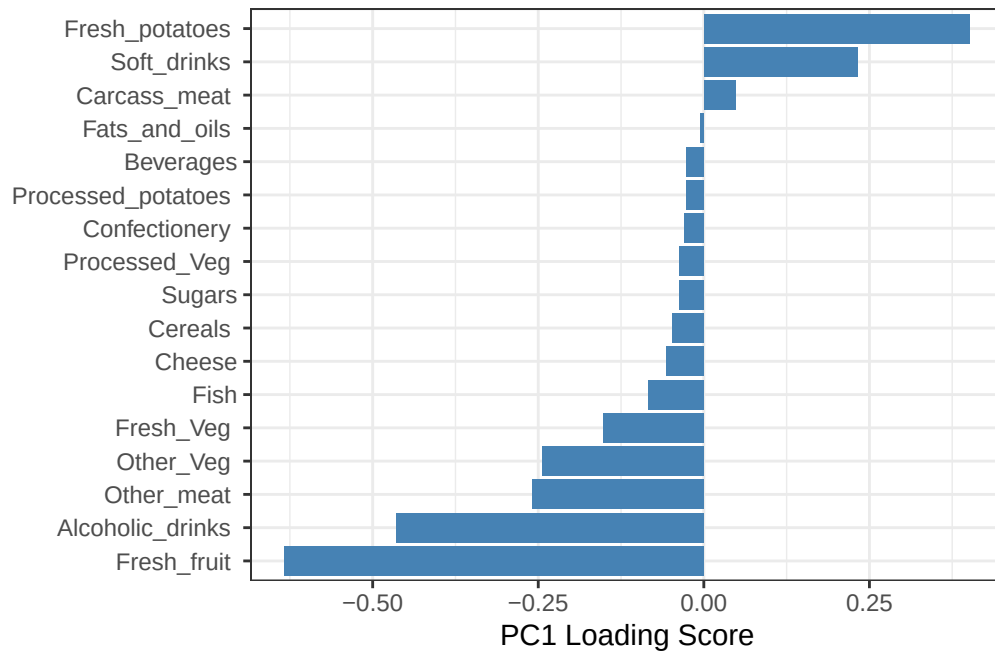


Digging Deeper (Variable Loadings)

```
ggplot(pca$rotation) + aes(PC1, row.names(pca$rotation)) + geom_col()
```



```
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar 'loadings plot' for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```

